

PENETRATION TEST REPORT

SQL Injection Vulnerability

CAIT — Continuous Assured Intelligent Testing

August 20, 2026

CRITICAL VULNERABILITY DISCOVERED

SQL Injection in User Lookup Form

Contents

1 Executive Summary

A **critical boolean-based blind SQL injection vulnerability** has been identified and successfully exploited in the DVWA application's user lookup functionality. The vulnerability allows complete database enumeration and credential extraction without authentication.

1.1 Impact

- Severity: CRITICAL (CVSS 9.8)
- Data Compromised: Admin credentials extracted
- Authentication Bypass: Possible
- Database Access: Full read access confirmed

1.2 Key Findings

- Admin password hash successfully extracted
- Plaintext recovered: password
- Extracted credentials validated via login
- All 5 database users enumerated

2 Vulnerability Details

2.1 Affected Component

Property	Value
URL	http://127.0.0.1/DVWA/vulnerabilities/sqli/
Method	POST
Parameter	id
Vulnerability Type	Boolean-Based Blind SQL Injection
Severity	CRITICAL

2.2 Technical Description

The application concatenates user input directly into SQL queries:

```
// Vulnerable Code Pattern
$id = $_POST['id'];
$query = "SELECT * FROM users WHERE id = '" . $id . "'";
$result = mysqli_query($connection, $query);
```

This allows arbitrary SQL injection and data extraction via boolean-based blind techniques.

3 Proof of Concept

3.1 Step 1: Enumerate All Users

Payload: `id=' OR '1'='1`
Result: Returns all 5 database users instead of single record.

3.2 Step 2: Boolean-Based Testing

TRUE Condition: `id=1' AND '1'='1` — Returns 1 record
FALSE Condition: `id=1' AND '1'='2` — Returns 0 records

3.3 Step 3: Password Hash Extraction

Character-by-character extraction using SUBSTRING comparison.
Extracted Hash: `5F4DCC3B5AA765D61D8327DEB882CF99`

3.4 Step 4: Verification

The extracted hash is the MD5 of: `password`
Verified via successful login with extracted credentials.

4 Exploitation Details

4.1 Attack Vector

- Type: Boolean-Based Blind SQL Injection
- Technique: Character-by-character extraction via response differential
- DBMS: MySQL
- Authentication Required: No
- Rate Limiting: Not detected

4.2 Data Extracted

Data	Value	Confirmed
Database	dvwa	YES
Table	users	YES
Username	ADMIN	YES
Password Hash	5F4DCC3B5AA765D61D8327DEB882CF99	YES
Plaintext	password	YES
Total Users	5	YES

5 Risk Assessment

5.1 CVSS v3.1 Score

Score: 9.8 (CRITICAL)
CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

5.2 Business Impact

- Credential Compromise: All user accounts at risk
- Authentication Bypass: Admin access obtainable
- Data Breach: Full database disclosure
- Regulatory Impact: Data protection violations
- Reputation: Severe impact on organizational trust

6 Remediation

6.1 Immediate Actions (Urgent)

1. Use Prepared Statements:

```
$stmt = $mysqli->prepare(
    "SELECT * FROM users WHERE id = ?"
);
$stmt->bind_param("i", $_POST['id']);
$stmt->execute();
```

2. Input Validation:

```
$id = intval($_POST['id']);
if ($id <= 0) die("Invalid ID");
```

6.2 Short-Term Actions (1-2 weeks)

- Restrict database account to SELECT-only operations
- Remove access to information_schema
- Implement error suppression for production
- Force password reset for all users

6.3 Long-Term Actions (Ongoing)

- Implement security code review process
- Deploy Web Application Firewall (WAF)
- Conduct regular SQL injection assessments
- Implement automated security scanning in CI/CD
- Perform quarterly penetration testing

7 References

- OWASP SQL Injection: https://owasp.org/www-community/attacks/SQL_Injection
- CWE-89: <https://cwe.mitre.org/data/definitions/89.html>
- WSTG: <https://owasp.org/www-project-web-security-testing-guide/>

8 Conclusion

A critical boolean-based blind SQL injection vulnerability has been confirmed in the DVWA application. Successful exploitation allows complete database enumeration and credential extraction. Immediate patching is required.

STATUS: CRITICAL - IMMEDIATE ACTION REQUIRED